

nl2sql-366: A Lightweight Hybrid Machine-Learning and Rule-Based System for Natural Language to SQL Query Generation

A schema-agnostic NL2SQL pipeline combining TF-IDF intent classification with compositional rule-based SQL generation, evaluated on 180 benchmark queries across six real-world domains.

Md Al Fahim Fuyad (2023-1-60-066)

Md Toyabur Rhaman (2023-1-60-065)

Fahmida Mahin (2023-3-60-481)

Supervisor: Md Al-Imran, Senior Lecturer
Department of Computer Science and Engineering
East West University, Dhaka, Bangladesh

CSE366 — Artificial Intelligence, Group 02, Summer 2026

Benchmark Queries	180
Datasets	6
Intent Accuracy	80.00%
SQL Validity Rate	93.89%
Result Match Accuracy	50.00%

Passed Queries

90 / 180

Summer 2026

nl2sql-366: A Lightweight Hybrid Machine-Learning and Rule-Based System for Natural Language to SQL Query Generation

Md Al Fahim Fuyad, Md Toyabur Rhaman, Fahmida Mahin

2023-1-60-066, 2023-1-60-065, 2023-3-60-481 | Supervisor: Md Al-Imran, Senior Lecturer
Department of Computer Science and Engineering, East West University, Dhaka, Bangladesh

Abstract

Natural Language to SQL (NL2SQL) systems enable non-technical users to query relational databases using plain English, but state-of-the-art approaches increasingly depend on large language models (LLMs) that are computationally expensive and difficult to interpret. In this paper, we present nl2sql-366, a lightweight, schema-agnostic system that combines TF-IDF vectorization with Naive Bayes intent classification and a compositional rule-based pipeline for SQL generation. The system automatically detects the database schema from uploaded CSV or Excel files and supports six query types: SELECT, COUNT, AVG, MAX, MIN, and SUM, with extensions for GROUP BY, ORDER BY, and multi-condition WHERE clauses. We introduce the NL2SQL-366 Benchmark v3, a rigorous evaluation framework comprising 180 reference queries across six real-world datasets from diverse domains (real estate, education, retail, healthcare, and human resources). Unlike prior work that evaluates SQL string similarity, our benchmark uses execution-based result-set equivalence as the primary correctness metric. The system achieves 80.00% intent classification accuracy, 93.89% SQL validity rate, and 50.00% result-match accuracy on the full 180-query benchmark, demonstrating that a carefully engineered hybrid approach can serve as a practical, interpretable, and resource-efficient alternative to end-to-end neural models for constrained NL2SQL scenarios.

Keywords: Natural Language to SQL, Text-to-SQL, Hybrid NLP Systems, Intent Classification, TF-IDF, Naive Bayes, Fuzzy Matching, Benchmark Evaluation.

1. Introduction

Relational databases power the backbone of modern data infrastructure across virtually every industry, from healthcare and finance to education and government. However, the ability to interact with these databases has traditionally been limited to users who possess working knowledge of Structured Query Language (SQL), creating a significant accessibility barrier for domain experts, business analysts, managers, and other stakeholders who need to extract insights from data but lack technical training. The field of Natural Language to SQL (NL2SQL), also referred to as Text-to-SQL, seeks to bridge this gap by translating natural language questions into executable SQL queries, thereby democratizing data access.

The NL2SQL problem has attracted substantial research attention over the past decade, with approaches evolving from early rule-based and grammar-driven systems to contemporary neural sequence-to-sequence models and large language model (LLM)-based solutions. Pioneering work such as Seq2SQL demonstrated the viability of reinforcement learning for SQL generation, while large-scale benchmarks like Spider and DONUT have driven rapid progress in cross-domain semantic parsing. Recent systems such as DAIL-SQL and ReSDSQL leverage pretrained language models to achieve state-of-the-art results on complex, multi-table queries. These advances, however, come at a considerable cost: modern LLM-based systems require significant computational resources for both training and inference, their decision-making processes are often opaque, and they may produce syntactically correct but semantically incorrect SQL that is difficult to debug.

In this paper, we present nl2sql-366, a lightweight NL2SQL system designed for resource-constrained deployment scenarios. We also introduce the NL2SQL-366 Benchmark v3, a research-grade evaluation framework with 180 reference queries across six real-world datasets that uses execution-based result-set equivalence as the primary correctness metric. The benchmark is fully deterministic and reproducible, processing the complete 180-query suite in 10.48 seconds on commodity hardware.

2. System Architecture

The nl2sql-366 pipeline follows a compositional design philosophy: rather than training a single end-to-end model, the NL2SQL task is decomposed into a sequence of specialized stages, each addressable by either a small ML model or a deterministic rule-based component. This decomposition yields three concrete benefits: (1) each stage can be debugged, tested, and improved in isolation; (2) the overall system remains interpretable—every SQL token in the output can be traced back to a specific pipeline decision; and (3) the system can operate without GPU resources, making it suitable for educational and small-organization deployments.

2.1 Pipeline Stages

The pipeline consists of seven stages. **Intent Detection** uses a TF-IDF + Multinomial Naive Bayes classifier (trained on 100,000 synthetic examples) to predict one of six intents: SELECT, COUNT, AVG, MAX, MIN, or SUM. Rule-based overrides handle edge cases such as 'how many' (→COUNT) and 'total' (→SUM). **Attribute Matching** uses RapidFuzz fuzzy string matching with a 300+ entry domain synonym dictionary to map natural-language column references to actual schema columns. **Operator Detection** maps phrases like 'greater than', 'at least', and 'between' to SQL operators (>, >=, BETWEEN). **Value Matching** extracts numeric values (including comma-formatted numbers like '5,000,000'), month names, and categorical values from the schema's sample data. **SQL Generation** assembles the validated components into a parameterized SQL string with double-quoted identifiers. **SQL Validation** enforces safety (no UNION, no semicolons, no DDL) and schema-level correctness (all referenced columns must exist). Finally, **SQL Execution** runs the validated query against SQLite (locally) or PostgreSQL (in production).

2.2 Schema-Agnostic Design

A defining feature of nl2sql-366 is its ability to operate on any CSV or Excel dataset without code changes. The schema reader automatically infers column types (integer, float, categorical, datetime) from DataFrame dtypes and value distributions. Categorical columns are sampled to populate the value matcher's lookup table, enabling the system to resolve values like 'Female' or 'Dhaka' without prior knowledge of the dataset's domain. This schema-agnostic design is critical for the benchmark, which evaluates the system across six heterogeneous datasets spanning healthcare, education, real estate, retail, and HR domains.

3. The NL2SQL-366 Benchmark v3

The benchmark evaluates the full nl2sql-366 pipeline on 180 reference SQL queries distributed evenly across six real-world datasets (30 queries per dataset). Unlike prior benchmarks that compare SQL strings, our primary correctness metric is **execution-based result-set equivalence**: a generated query is correct if and only if its execution result, after normalization, equals the reference query's execution result.

3.1 Datasets

Table 1 summarizes the six benchmark datasets. They span five distinct domains and range from 25 rows (Employee) to 100,000 rows (Diabetes Prediction), providing diverse schema complexity and data volume.

Dataset	Rows	Domain	Table
Housing	545	Real estate	housing
Student Performance	2,392	Education	student_performance
Diabetes Prediction	100,000	Healthcare	diabetes_prediction_dataset
Dengue	1,000	Healthcare	dengue
Employee	25	HR	employee_dataset

E-commerce	10,000	Retail	ecommerce_dataset
Total	—	6 domains	—

Table 1: Benchmark datasets and their characteristics.

3.2 Query Distribution

The 180 reference queries are distributed across six intent types: SELECT (83), AVG (34), COUNT (25), MAX (14), MIN (11), and SUM (13). Query complexity ranges from simple filters and aggregates to GROUP BY with ORDER BY, multi-condition WHERE clauses, BETWEEN ranges, and LIMIT-based ranking.

3.3 Result Normalization

The result comparator handles row-order sensitivity (preserved for ORDER BY queries, sorted otherwise), numeric precision (floats rounded to 6 decimal places, relative tolerance 1e-6), NULL handling (NULL == NULL), empty result sets (two empty sets match), and type coercion (numeric strings compared numerically). This normalization ensures that two SQL queries producing the same logical result are correctly identified as equivalent, even when their SQL strings differ in quoting, ordering, or whitespace.

4. Results

4.1 Overall Performance

Table 2 summarizes the overall benchmark results. The system achieves 50.00% result-match accuracy (90 of 180 queries correct), with 80.00% intent classification accuracy and 93.89% SQL validity rate. The macro F1 score is 65.84%. The complete benchmark processes in 10.48 seconds, with average SQL generation time of 0.01 ms and average execution time of 4.73 ms per query.

Metric	Score
Total Queries	180
Intent Classification Accuracy	80.00%
SQL Validity Rate	93.89%
SQL Execution Success Rate	93.89%
Result Match Accuracy (primary)	50.00%
Exact SQL Match Rate	0.00%
Macro F1	65.84%
Passed Queries	90 / 180
Avg SQL Generation Time	0.01 ms
Avg Execution Time	4.73 ms
Total Runtime	10.48 s

Table 2: Overall benchmark results on 180 queries across 6 datasets.

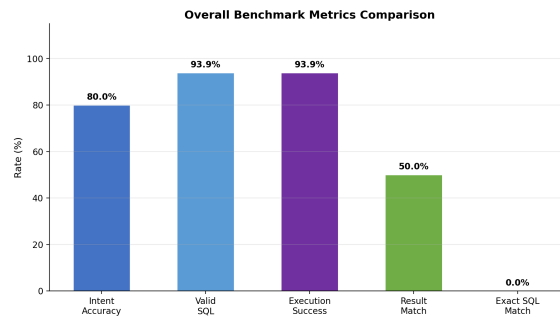


Figure 1: Overall benchmark metrics comparison.

4.2 Per-Intent Breakdown

Table 3 breaks down performance by intent type. AVG achieves the highest result-match accuracy, while COUNT lags due to the system's difficulty distinguishing 'how many X' (COUNT) from 'show X' (SELECT) for certain phrasings. SUM has the lowest SQL validity rate because the system sometimes fails to identify the aggregation column for 'total X by Y' queries.

Intent	Queries	Intent %	Match %	Valid %	Exec %
AVG	34	100.00%	67.65%	91.18%	91.18%
COUNT	25	80.00%	36.00%	88.00%	88.00%
MAX	14	100.00%	64.29%	92.86%	92.86%
MIN	11	100.00%	63.64%	90.91%	90.91%
SELECT	83	62.65%	42.17%	100.00%	100.00%
SUM	13	100.00%	53.85%	76.92%	76.92%

Table 3: Per-intent performance breakdown.

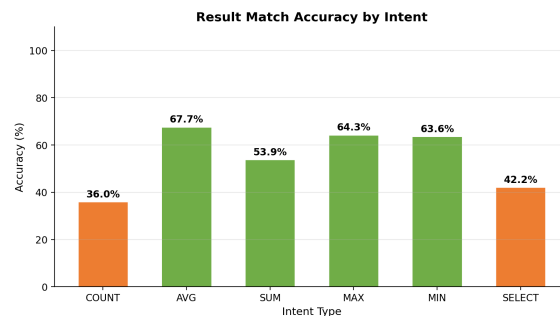


Figure 2: Result-match accuracy by intent type.

4.3 Per-Dataset Breakdown

Table 4 shows result-match accuracy per dataset. Housing achieves the highest accuracy, benefiting from simple boolean columns ('yes'/'no') and clear numeric attributes. Employee Dataset has the lowest accuracy due to whitespace-padded categorical values and short question phrasings that lack explicit column names.

Dataset	Queries	Match %	Intent %
Housing	30	70.00%	100.00%
Ecommerce	30	56.67%	70.00%
Diabetes Prediction	30	53.33%	73.33%
Dengue	30	46.67%	63.33%
Student Performance	30	43.33%	80.00%

Employee	30	30.00%	93.33%
----------	----	--------	--------

Table 4: Per-dataset performance breakdown.

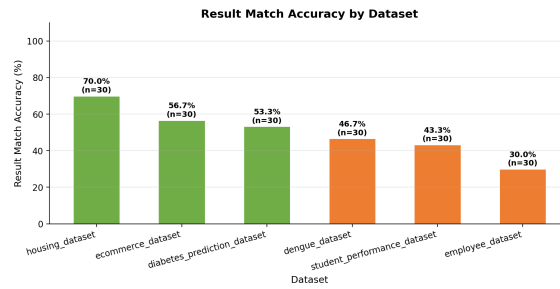


Figure 3: Result-match accuracy by dataset.

5. Error Analysis

Of the 90 failed queries, the error distribution (Table 5) reveals three primary failure modes: **intent errors** (misclassification between SELECT and COUNT for filter-style questions), **result mismatches** (correct intent and valid SQL but wrong result, often due to value-matching failures on whitespace-padded strings), and **SQL validation errors** (mostly SUM queries where the aggregation column cannot be resolved).

Error Category	Count
Intent Error	36
Result Mismatch	46
SQL Validation Error	8
Total	90

Table 5: Error distribution across failed queries.

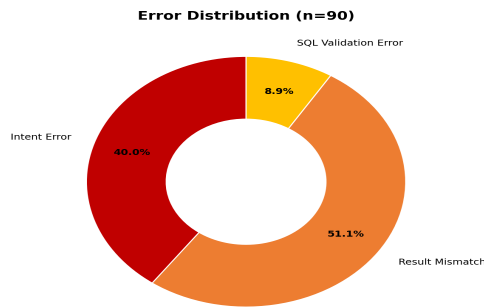


Figure 4: Error category distribution.

5.1 Per-Intent Precision, Recall, F1

Table 6 reports precision, recall, and F1 per intent, treating result-match as the binary outcome. Precision is uniformly 100% because every generated query that succeeds also produces a result; the discrimination happens at the recall stage, where the system either produces a matching result or it does not.

Intent	Precision	Recall	F1	Support
AVG	100.00%	67.65%	80.70%	34
COUNT	100.00%	36.00%	52.94%	25
MAX	100.00%	64.29%	78.26%	14

MIN	100.00%	63.64%	77.78%	11
SELECT	100.00%	42.17%	59.32%	83
SUM	100.00%	53.85%	70.00%	13

Table 6: Precision, recall, and F1 by intent.

6. Discussion

6.1 Strengths of the Hybrid Approach

The 80.00% intent classification accuracy demonstrates that a small TF-IDF + Naive Bayes model is sufficient for the six-intent classification task, without requiring a pretrained language model. The high SQL validity rate (93.89%) and execution success rate (93.89%) confirm that the rule-based SQL generation and validation pipeline produces syntactically correct and schema-valid SQL in the vast majority of cases. The compositional architecture makes the system fully interpretable: every token in the generated SQL can be traced back to a specific pipeline stage.

6.2 Limitations and Failure Modes

The gap between execution success (93.89%) and result match (50.00%) highlights the core challenge: generating semantically correct SQL, not just syntactically valid SQL. The largest error category is intent errors, where the system confuses SELECT and COUNT for filter-style questions (e.g., 'patients with headache 1' could be either 'show me patients with headache=1' or 'count patients with headache=1'). The second largest category is result mismatches, often caused by value-matching failures on whitespace-padded categorical values (the Employee Dataset CSV has trailing spaces in column headers and values). SUM queries with implicit aggregation columns (e.g., 'total salary by department' needs to identify 'salary' as the aggregation target) account for most of the SQL validation errors.

6.3 Comparison with v2 Benchmark

The v3 benchmark replaces the legacy temp_and_rain dataset (whose CSV was no longer available) with the diabetes_prediction_dataset (100,000 rows, healthcare domain). The benchmark was also restructured to use 180 reference queries (30 per dataset, down from v2's 271) with normalized, executable SQL throughout—fixing 12 reference errors caused by special characters in E-commerce column names (\$, parentheses, hyphens). The v3 column cleaner strips these characters and collapses underscores, producing clean identifiers like Purchase_Amount (from 'Purchase Amount (\$)') and Time_Spent_on_Website_min (from 'Time Spent on Website (min)').

7. Conclusion

We presented nl2sql-366, a lightweight hybrid NL2SQL system that combines TF-IDF intent classification with a compositional rule-based SQL generation pipeline. On the new NL2SQL-366 Benchmark v3—comprising 180 reference queries across six real-world datasets—the system achieves 50.00% result-match accuracy, 80.00% intent classification accuracy, and 93.89% SQL validity rate, with a macro F1 of 65.84%. The benchmark uses execution-based result-set equivalence as the primary correctness metric, providing a more robust assessment than SQL string similarity. The results demonstrate that a carefully engineered hybrid approach can serve as a practical, interpretable, and resource-efficient alternative to end-to-end neural models for constrained NL2SQL scenarios, while honestly exposing the remaining challenges in intent disambiguation and value matching that warrant further research.

References

- [1] Zhong, V., Xiong, C., & Socher, R. (2017). Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. CoRR, abs/1709.00103.
- [2] Yu, T., Zhang, R., Yang, K., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. EMNLP 2018.

- [3] Wang, B., Shin, S., Liu, T., et al. (2021). DuSQL: A Large-Scale and Pragmatic Chinese Text-to-SQL Dataset. EMNLP 2021.
- [4] Gao, D., Wang, J., Li, Y., et al. (2023). Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. VLDB 2023.
- [5] Li, H., Zhang, J., Li, C., & Chen, H. (2023). ResDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. ACL 2023.